Linux Fest 2026

How to Debug a Bug You Cannot Reproduce

Systematic Investigation, Observability, and System Analysis

Hooriyeh Maaleki



It’s 3 AM. Your phone rings.

“The service is hanging again.”

You SSH in. Everything looks fine.

You restart it. It works.

The logs show nothing.

What do you do?



The Uncomfortable Truth

Most non-reproducible bugs cannot be solved immediately.

Not because you’re not good enough.

Because the observability wasn’t built in advance.

Stop Asking the Wrong Question

Most post-mortems start here:

“Why can’t I reproduce this?”

That question sends you in circles. Try these instead:

- ✓ “What evidence do I already have?”
- ✓ “What was I not collecting when it happened?”
- ✓ “What do I need to build so I’m ready next time?”

The bug already happened. Your job now is investigation, not re-enactment.





The Socratic Shift

Before touching any code, ask three questions:

- ✓ What do we know?
  - ✓ Anchors you to facts, not guesses
- ✓ What do we not know?
  - ✓ Maps your blind spots
- ✓ What are we assuming?
  - ✓ Exposes the most dangerous traps

Most debugging sessions fail because teams skip to step 3 and treat assumptions as facts.





Firefighting vs Engineering

🔥 Firefighting
Chasing every problem as it appears
You jump into action cold
You wait for the next fire
Core concern: “How do I fix this?”

⚙️ Engineering
Building systems that speak for themselves
The data was already being collected
You improve the system to prevent recurrence
Core concern: “How do I stay ready?”

Both modes are real but the goal is to spend more time in the right column.

The Mindset Shift

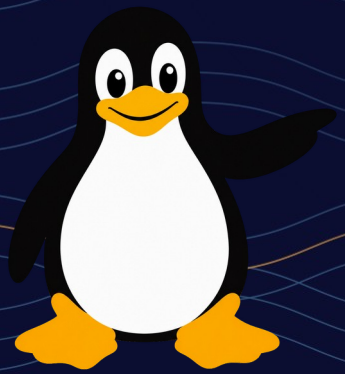
Reactive Engineer	Proactive Engineer
“Let me try to reproduce it”	“Let me examine the evidence”
Adds logs after the incident	Designs observability before it
Fixes the symptom	Fixes the root cause + prevents recurrence
Dreads the 3 AM call	Has the data ready when it comes

- The goal isn’t to reproduce the bug. The goal is to never be blind when it strikes.





Understanding the Problem



Why Bugs Resist Reproduction

Some bugs don’t show up on demand — and that’s not random. There are real structural reasons why:

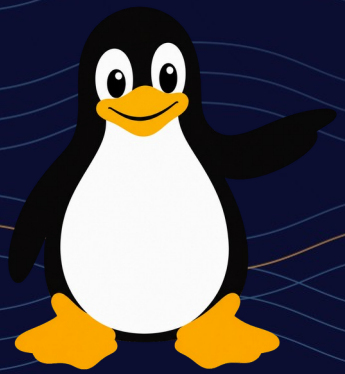
- Timing & ordering dependencies
  - Modern systems run concurrent threads, async callbacks, interrupt handlers.
- Race conditions
  - Two code paths that share state without proper synchronization will usually work fine.
- Environment differences (dev vs prod)
  - Dev has 16 GB RAM, SSDs, a single process, and no other tenants. Prod has memory pressure, a noisy NIC, a slow NFS mount, and 40 other processes fighting for CPU.
- Resource exhaustion
  - File descriptors, memory, threads, socket connections
- Transient failures (network, hardware)
  - A packet that arrives 400ms late. A memory cell that flips under thermal load. A disk sector that reads correctly 99.9% of the time.

Taxonomy of Non-Reproducible Bugs

Understanding which type you’re dealing with shapes your entire investigation strategy.

- Timing-dependent
  - Race condition on shared queue
  - Lock ordering, memory barriers, thread scheduling logs
- Environment-dependent
  - Works on dev, fails on prod
  - Kernel version, ulimits, filesystem type, env vars, library versions
- Load-dependent
  - Only under high CPU/memory
  - Resource metrics at time of failure, profiler output, OOM events
- State-dependent
  - Triggered by specific sequence
  - Call history, audit logs, state machine transitions
- Hardware-dependent
  - Flaky NIC, memory error
  - dmesg, ECC error counters, mcelog, NIC stats, SMART data

Debugging Mechanisms



The Debugging Toolbox

Debugging is not one tool — it’s a decision about what kind of evidence you need.

- Logging
  - What happened, in what order, with what context?
- Interactive debugger
  - What is the exact state at this point in execution?
- Core dumps / crash reports
  - What was the program doing when it died?
- System call tracing
  - How is the process interacting with the OS?
- Profiling
  - Where is time or memory actually going?
- Static analysis
  - What does the compiler or linter know that you missed?
- Dynamic analysis (sanitizers)
  - What undefined behavior is hiding at runtime?
- Metrics & observability
  - Is this a new problem, or has it been slowly growing?

The right tool depends on two things: what you have access to, and what kind of bug you’re chasing.

Interactive Debuggers

An interactive debugger gives you a live window into a running or crashed process. The exact interface varies but the concepts are universal:

- Breakpoints
  - stop execution at a point of interest
- Watchpoints
  - stop when a value changes
- Backtraces
  - see the full call chain at the moment of failure
- Thread inspection
  - examine all threads, not just the one that crashed
- Variable inspection
  - read memory and data structures in their current state

Most useful for: bugs you can reproduce, post-crash analysis with a core dump, understanding unfamiliar code paths.
Least useful for: bugs that disappear the moment you attach.

Core Dumps and Crash Reports

A crash report is a snapshot of the process at the moment it died. Depending on the platform:

- Unix/Linux: core dump file / analyzed offline with GDB/LLDB
- Windows: minidump / Visual Studio
- Mobile / managed runtimes: structured crash report with stack trace and register state

The key idea: you don’t need to reproduce the crash to diagnose it. The state is preserved. You just need to ask it the right questions. What to look for:

- Which thread faulted, and why
- The full backtrace, including all frames
- The values of key variables at each frame
- Whether other threads were in a suspicious state

The limiting factor is always symbol information. Stripped binaries give you addresses, not names. Keep your symbol files.

System Call Tracing

A system call tracer sits between your process and the kernel, recording every interaction:

- file opens
- network I/O
- memory mappings
- signals
- process management

Tools: strace, ltrace for library calls.

This is useful when:

- The bug is in the interaction between your program and the OS, not in your logic
- You’re debugging a binary you don’t have source for
- Something is blocking, failing, or taking longer than expected at the syscall level

What to look for: failed syscalls (ENOENT, EACCES, EAGAIN), unexpectedly long gaps between calls, signals arriving at bad times, unexpected file or network activity.

The 3-Question Decision Framework

Before reaching for any tool, answer three questions:

Can I reproduce it right now?
Yes: attach an interactive debugger or add targeted instrumentation and run it.
No: you need observability, not a debugger.

Did it crash?
Yes: start with the crash report or core dump. The evidence is already there.
No: it misbehaved silently, which is harder. You need logs and metrics.

Does it only happen in production?
Yes: you cannot debug it interactively. Your only leverage is the data you already instrumented before it happened.

These three questions alone will save you from the most common mistake: reaching for printf or a debugger when the real problem is that you have no data.

Symptom → Tool Decision

- Process crashed
  - Core dump + debugger
- Hang or deadlock
  - Attach debugger, inspect thread states; syscall tracer
- Memory corruption
  - Dynamic analysis (ASan, Valgrind, sanitizers)
- Slow / high CPU
  - Profiler (perf, gprof, sampling profilers)
- Intermittent failure
  - Structured logging + metrics
- Only in production
  - Observability: logs, metrics, distributed tracing
- Wrong output, no crash
  - Unit tests + interactive debugger on the failing case
- Works locally, fails in CI/staging
  - Environment diff: deps, config, OS version, resource limits

Language and Runtime Constraints

Every language and runtime has failure modes that shape which tools work:

Native code (C, C++, Rust unsafe):
No runtime reflection: you see addresses, not types, without symbols
Undefined behavior is silent until it isn't
Compiler optimizations reorder and eliminate code, making debuggers disorienting
Memory errors can corrupt state silently long before the crash

Managed runtimes (JVM, .NET, Python, Go):
GC pauses and finalizer timing affect timing-sensitive bugs
Exceptions may be swallowed silently at framework boundaries
Thread models vary: coroutines and async make stack traces harder to read
Heap dumps are the equivalent of core dumps

The universal constraint: debugging tools observe the process, and observation changes timing. The closer you look, the more some bugs hide.

OS and Environment Constraints

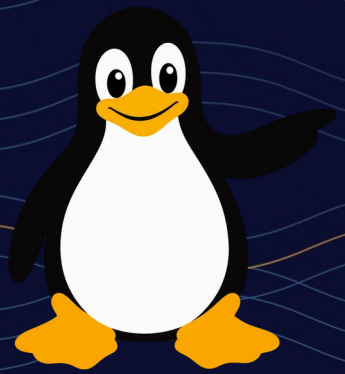
The application is not the whole system. Bugs often live in the gap between the two process:

Signal handling is asynchronous: a signal can arrive between any two instructions
File descriptor and socket limits are process and system-wide, not just application-wide
Memory pressure causes different failure modes depending on overcommit settings and the OOM killer
Filesystem behavior differs between local, NFS, tmpfs, and memory-constrained environments
Clock drift and time jumps affect timeouts, leases, and any logic that compares timestamps

Tools at this layer:
dmesg, journalctl, vmstat, iostat, ss, lsof, netstat, top, /proc

The habit to build:
when a bug resists all application-level explanations, look one layer down.

Observability



What Is Observability?

Observability answers one question:
“Can I understand what my system is doing from the outside, just by looking at its outputs?”

Monitoring tells you something is wrong.
Observability tells you why.

A system is observable when you can:

- Ask arbitrary questions about its internal state without shipping new code
- Reconstruct what happened before a failure using existing data
- Understand behavior you didn’t predict in advance

Why it matters for unreproducible bugs:
A bug you can’t reproduce is just a bug you don’t have enough data about. Observability is the discipline of collecting that data before the bug happens.

Observability isn’t a tool you add after things break. It’s an engineering decision you make before they do.

Monitoring vs Observability

They’re not the same thing. The difference matters when you’re hunting a bug you can’t reproduce.

	Monitoring	Observability
Question it answers	Is something broken?	Why is it broken?
Approach	Pre-defined checks on known failure modes	Open-ended exploration of unknown states
Data model	Fixed dashboards, alerts on thresholds	Rich, queryable telemetry you can slice any way
When you set it up	After you know what to watch	Before you know what will go wrong

The Key Question

“If this bug happens tomorrow at 3 AM, what data do I need to diagnose it without touching the system?”

What that question forces you to think about:

- State before the failure:
  - What was the system doing in the 60 seconds leading up to the crash?
- Inputs that triggered it:
  - Which request? Which user? Which config version was live?
- Resource context:
  - Memory pressure? Open file descriptors? CPU saturation?
- Concurrency state:
  - Which threads were active? What locks were held?
- Environment snapshot:
  - Kernel version, loaded modules, active network connections, ulimits

The discipline:
Don’t wait for the bug to happen, then scramble to add logging. Ask the question now, while you’re writing the code and build the instrumentation that answers it.

What Can We Add for Observability?

These are the practical additions you can make to a real system from zero observability to full situational awareness.

- Structured Logging
  - Replace free-form strings with structured key-value output.
- Metrics with Prometheus / StatsD
  - Expose counters, gauges, and histograms your monitoring system can scrape
- Health & Readiness Endpoint
  - A /healthz endpoint that returns structured JSON with subsystem status.
- Ring Buffer for Recent Events
  - An in-memory circular buffer of the last N operations, flushed to disk on crash.
- Signal Handlers for Live Inspection
  - Define a specific function after catch a special signal
- Distributed Tracing (if applicable)
  - If the system spans services or processes, attach a trace_id to every operation and propagate it.
- Crash Reports with Context
  - On fatal signal (SIGSEGV, SIGABRT), capture:
  - Stack trace / Last N log entries from ring buffer / Resource snapshot at time of crash / Active configuration version

Bad Log vs Structured Log

Error occurred
Something went wrong in operation1
Failed
DEBUG: here
DEBUG: here2
DEBUG: done

Bad Log vs Structured Log

Field category	Example key	Purpose
Time	ts=5ms	Sorting, correlation
Severity	level=7	Filtering
Component	component=	Where to look
Message	msg=Fixed	human-readable label
Entity identity	device_id=1	Which object
Version	config_version=1.1.2	Reproducibility
Error code	errno=-5124	Exact failure cause
Duration	duration_ms=1	Performance signal
Trace ID	session_id=ad-34-qg	Correlation across lines

Metrics vs. Logs

Logs answer **“What happened?”.** they give you high detail, high volume data where every event gets its own record. Their primary use is post-mortem analysis: you filter by field, grep by trace ID, and correlate across components to reconstruct the exact sequence of events that led to a failure. The tradeoff is retention cost. every log line is a row, making them expensive to store and query at scale. Without structure, it’s also hard to spot patterns in the noise.

Metrics answer **“How often / how much?”.** they’re low cardinality, aggregated numbers tracked over time. A single counter or gauge can compress thousands of events into one value. Their primary use is alerting and trending: you graph them over time, set threshold alerts, and detect rate-of-change anomalies before they become incidents. The retention cost is low, but the blind spot is loss of context.

The pattern: Metrics tell you when to investigate. Logs tell you what went wrong. You need both.

Correlation Ids: One ID The Whole Story

Generate a unique ID at the start of every operation. Attach it to every log line. Pass it through every subsystem call.

When a bug happens:

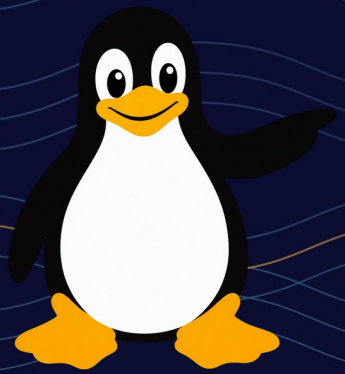
```
grep session_id=******* /var/log/app.log
```

Every log line from that one operation, in order, no noise, no guessing.

Rules:

- Generate at entry, pass forward, never from a global
- Include in every log line, IPC message, and error report
- Format doesn’t matter. a counter works as well as a UUID
- The discipline matters more than the tool

System Investigation



Look Beyond Your Code

Your code runs in a system. The system has limits.
Most bugs labeled “application bugs” are actually system interactions.

What to check beyond your code:

- Kernel & OS:
  - scheduler jitter, OOM killer, signal delivery latency
- File system:
  - disk full, inode exhaustion, NFS stale handles, sync delays
- Network stack:
  - retransmits, buffer exhaustion, MTU mismatches, TIME_WAIT storms
- Memory:
  - fragmentation, swap pressure, allocator behavior under load
- Hardware:
  - thermal throttling, bad RAM, ECC errors, clock drift
- Shared resources:
  - other processes on the same host, noisy neighbors in containers

The mindset shift

Don’t ask “is my code wrong?” first.

Ask “what was the system doing when this happened?”

Your code might be correct and still fail because the floor moved under it.

Tools for System Investigating

CPU

```
top -H -p <pid>          # per-thread CPU
perf top                 # live profiling
cat /proc/<pid>/stat     # raw stats
```

Look for: unexpected spikes, one thread pegged at 100%

Memory

```
free -h
cat /proc/<pid>/status | grep VmRSS
valgrind --leak-check=full ./daemon
```

Look for: growing RSS, swap usage, OOM events in dmesg

Tools for System Investigating

File Descriptors

ls /proc/<pid>/fd | wc -l
lsof -p <pid>
cat /proc/sys/fs/file-max
The limit is per-process (ulimit -n) and system-wide.

Network

ss -tnp
ss -s # summary
netstat -i # interface stats
Look for: stuck connections, TIME_WAIT accumulation, retransmits

Tools for System Investigating

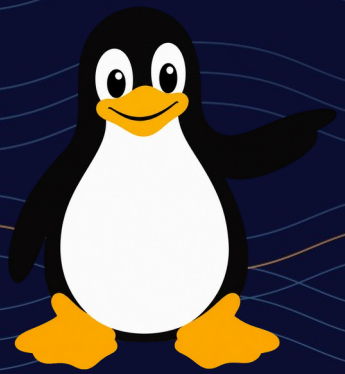
Disk I/O

bash
iostat -x 1
iotop -p <pid>
Look for: high await, 100% util, blocking writes

Kernel Messages

bash
dmesg -T | tail -50
journalctl -k --since "1 hour ago"
Look for: OOM kills, filesystem errors, NIC errors, segfaults

Hypothesis-Driven Investigation



From Evidence to Hypothesis

Debugging is a scientific process — not guesswork.

Evidence → Pattern → Hypothesis → Test → Confirm or Reject

A hypothesis must be falsifiable. This is a principle from the philosophy of science, associated with Karl Popper: “a claim is only a scientific hypothesis if you can design a test that could prove it wrong.”

The same applies in debugging:

- Weak hypothesis:
  - “Maybe there’s a bug somewhere”
  - “Could be a network issue”
  - “Probably a concurrency problem”
- Strong hypothesis:
  - “The FD count grows monotonically during backup operations”
  - “If latency on eth0 exceeds 200ms, this timeout fires”
  - “With two threads sharing the same session_id, the race is reproducible”

From Evidence to Hypothesis

Why this matters:
A falsifiable hypothesis tells you:

- Exactly what to measure
- What result would prove it wrong
- It guards against confirmation bias, the biggest enemy of good debugging

The practical path:

Observation: “The service crashes every night at 2 AM”
↓
Pattern: “The backup job starts every night at 1:50 AM”
↓
Hypothesis: “The backup job leaks resources until the OOM killer fires”
↓
Test: “Can I reproduce it with the right conditions?”
↓
Result: Confirm or reject. either one moves you forward

Common Failure Patterns

Each pattern has a signature you can measure. If you know what to look for, you stop guessing.

Patterns:

1) Resource leak: Monotonically growing counter
2) Deadlock: Two threads waiting on each other
3) Race condition: Failure rate scales with load
4) Unbounded queue: Memory grows under backpressure

What these have in common:

- They all look intermittent until you measure the right thing.
- A leak doesn’t crash immediately, it crashes after N hours
- A deadlock doesn’t always trigger, only when two threads interleave a specific way
- A race isn’t obvious at low load, the window is too small
- An unbounded queue is fine until the consumer falls behind
- The fix for all of them starts the same way: instrument the counter that exposes the pattern, then watch it over time. The signature tells you what to watch.

Turning an observation into a hypothesis

An observation is not a hypothesis.
“It only hangs after N hours” is an observation.
A hypothesis is testable: “The session table grows without bound because entries are never evicted, after ~8 hours at normal load it exceeds available memory.”

That hypothesis tells you exactly what to measure and what result would falsify it.

The pattern for each observation type:

- “Only after N hours”

hypothesis: X accumulates at rate R; at N hours it hits threshold T
test: measure X every 30 min; plot the slope; does it hit T at ~N hours?

- “Only under load”

hypothesis: resource Y becomes the bottleneck above Z req/s
test: stress test with gradually increasing load; watch Y; find the knee

- “Only in production”

hypothesis: prod has config/data property P that dev lacks
test: diff every env variable, limit, and dataset characteristic between envs

Practical Rule

A practical checklist before writing a hypothesis:

1) Can you name the specific counter or state that would confirm it?
2) Can you describe what the counter would look like if the hypothesis is wrong?
3) Does it rule out at least one other explanation?

If yes to all three, it’s a real hypothesis. If not, keep asking why until you get there.

Where Testing Enters the Loop

Tests Are Hypothesis Validators, Not Bug Finders

- Unit test
  - validates isolated logic after hypothesis points to a specific function
- Integration test
  - validates the interaction your hypothesis named
- Load/Stress test
  - validates the condition your hypothesis requires (concurrency, resource limit)
- Sanitizers
  - runs your hypothesis past the compiler’s memory model

The wrong way:
Run all the tests and hope one fails.

The right way:
Form a hypothesis first, then pick the layer that can falsify it.

Unit Tests in Hypothesis Validation

Unit tests are a scalpel, not a net. They confirm a hypothesis about isolated logic once you know what you're looking for.

What unit tests can confirm:

- A specific code path produces wrong output given a crafted input
- A resource is not released after an operation
- A counter increments past a boundary without guard
- A null pointer is reachable under a specific sequence of calls
- These are things you already have a hypothesis about. The test pins the fix and prevents regression.

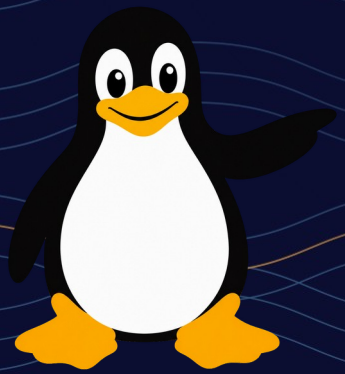
The right mental model:
Production observation
↓
System-level investigation (logs, metrics, dmesg, /proc)
↓
Narrow hypothesis: "function X corrupts state when input Y is negative"
↓
Unit test that forces input Y negative ← test lives here
↓
Fix confirmed by test passing

Practical Rule

Another practical rule:

- If you can write a unit test that fails before the fix and passes after it, you've understood the bug precisely enough to fix it correctly. That test is worth keeping forever.
- If you can't write that test, you haven't isolated the root cause yet so go back to the system.

Review



The Full Picture: Observability, Hypothesis, Testing, One Loop

OBSERVABILITY
Structured logs / Metrics / Ring buffer
↓ Evidence

HYPOTHESIS
Falsifiable claim about a specific component. Names the diagnostic tool that would disprove it
↓ Test to falsify or confirm

TESTING
Pick the right layer (unit / integration / sanitizer) the one your hypothesis demands
↓ Confirmed

REGRESSION TEST
Locks the fix permanently

Every commit is a check against every known ghost.

The bug you couldn’t reproduce wasn’t random. It was waiting for the right observation.

The Debugging Checklist

1) Collect all available evidence first
2) Check system resources (FDs, memory, CPU, disk)
3) Read kernel logs (dmesg, journalctl)
4) Form a falsifiable hypothesis
5) Test the hypothesis, don’t just fix the symptom
6) Write a regression test
7) Add observability to prevent recurrence
8) Document what you found and how

Key Takeaways

1) Most unreproducible bugs leave evidence, you need to know where to look
2) System investigation comes before code investigation
3) Structured logs with context are worth 10x unstructured logs
4) A hypothesis must be falsifiable
5) The fix isn’t done until there’s a regression test and a metric

The Mindset

1) Stop asking: “Why can’t I reproduce this?”
2) Start asking: “What would I need to diagnose this blind?”
3) Build observability before you need it.
4) Investigate evidence before you guess.
5) Fix root causes, not symptoms.

Further Reading & References

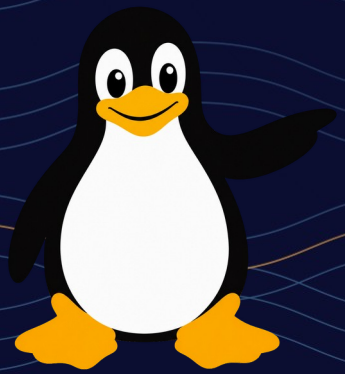
Books
Debugging: The 9 Indispensable Rules for Finding Even the Most Elusive Software and Hardware Problems, David J. Agans
The Pragmatic Programmer, Hunt & Thomas
Systems Performance: Enterprise and the Cloud, Brendan Gregg
Site Reliability Engineering, Google

Tools & Docs
GDB Documentation
Valgrind User Manual
strace Manual
GCC Sanitizer Docs
Linux perf Wiki

Articles & Talks
Brendan Gregg, Linux Performance Analysis in 60 Seconds, brendangregg.com
Julia Evans, How does strace work?, jvns.ca
Dan Luu, Debugging stories, danluu.com/debugging-stories
Cindy Sridharan, Monitoring and Observability, medium.com/@copyconstruct

Standards & Specifications
POSIX.1-2017 (errno codes, signal handling), pubs.opengroup.org
RFC 5424, The Syslog Protocol (structured log format)
OpenTelemetry specification, opentelemetry.io/docs/specs

Thanks for listening.



Contact Info:

Email: hooriyeh.maaleki@gmail.com

Telegram: @libhmlk

LinkedIn: Hooriyeh Maaleki

